```
function forceDirectedEmbeddingWithRestart(graph, alpha, runs,
maxSteps)      #1
  bestCost ← inf
  for _ in {1..runs} do
                                                #2
    (x,y) ← forceDirectedEmbedding(graph, alpha,
maxSteps)                      #3
    if cost(graph,x,y) < bestCost
then                                             #4
      bestX, bestY, bestCost ← (x, y, cost(graph,x,y))
  return (bestX,bestY)
```

This concludes our discussion on gradient descent; in the next chapters, we will explore alternative algorithms for optimization of cost-based solutions.

We'll use the rest of this chapter, instead, to catch up with the methods to compute edge crossing in embeddings.

16.6 Summary

- **Cost functions**: Many problems, including many in machine learning, are based on defining a proper cost function that measures how good a solution is, and then running an optimization algorithm to try and find the solution with minimal cost.
- **Algorithms**: there is a plethora of algorithms that can be used for this purpose: from random sampling, where we just thoughtlessly pick a solution and measure how good it is, to local and global optimization algorithms, like hill descent (which uses a random local approach) and gradient descent (which, instead, leverages prior knowledge about the function to be optimized).
- **Gradient descent**: This is a local optimization algorithm: usually it starts by selecting a random solution, and then it